CONAHCYT

CIMAT Centro de Investigación en Matemáticas, A.C.

# Algoritmos y optimización de código

Unidad 2. Algoritmos básicos

Diplomado en **Cómputo de Soluciones avanzadas**

# Introducción a la complejidad

- Divide y vencerás
- Algoritmos voraces
- Programación dinámica
- Algoritmos paralelos

# Link github

- https://github.com/trejoel/DiplomadoCAS-IntroAlgoritmos

Python

↓

google Colab

# Ejercicio de programación

[2, -6, 8] = 4

- Una Fintech presenta los precios en dólares de cierta criptomoneda
  - Ejemplo $[-4, 2, -6, 8, -2, 4, 2, -10, 8]$
- Se desea identificar los índices inferior y superior en que hubiese convenido adquirir la criptomoneda; es decir, aquella que maximice la ganancia
- ¿Cómo implementarías este algoritmo?
  - ¿Cúal es el tiempo de ejecución de este algoritmo?
  - Contar cuántas operaciones se realizan

# Problema del subarreglo máximo

- El problema del subarreglo máximo consiste en que dado un arreglo $A = [1..n]$ encontrar el subarreglo contiguo de $A$ tal que la suma sea la mayor posible.

- Considere el ejemplo del siguiente arreglo:

  - $A = [-4, 2, -6, 8, -2, 4, 2, -10, 8]$

- ¿Cuál sería el subarreglo de

[i, i]

MAX

[1, 1]
[1, 2]
[1, 3]
[1, N]

[2, 2]
[2, 3]
[2, N]

[3, 3]
[3, 4]

[8, 8]

# Problema del subarreglo máximo

- El problema del subarreglo máximo consiste en que dado un arreglo $A = [1..n]$ encontrar el subarreglo contiguo de $A$ tal que la suma sea la mayor posible.

left_sum ← -∞
sum ← 0
for i=mid, low
sum ← sum + A[i]
if sum > left_sum
left_sum ← sum

- Considere el ejemplo del siguiente arreglo:

mid

- $A = [-4, 2, -6, 8, -2, 4, 2, -10, 8]$

left_sum ← 6
right_sum = 4 ✗ 6

- ¿Cuál sería el subarreglo de
  - Resp: [8,-2,4,2]

# Problema del subarreglo máximo

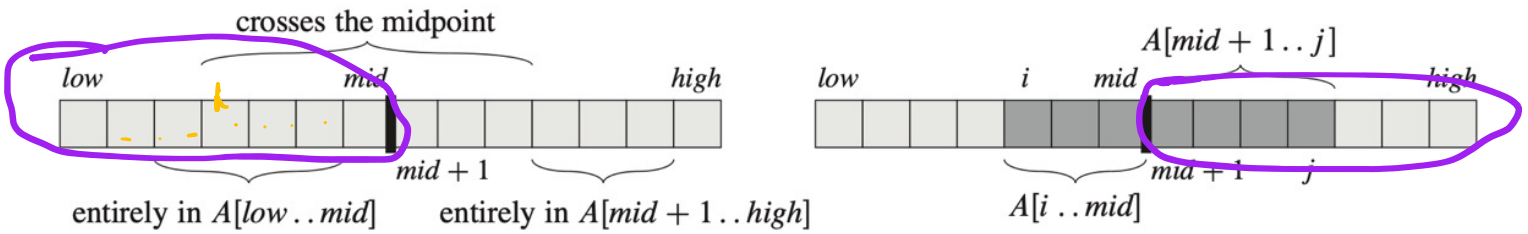
- Fuerza bruta
  - ¿Cuál es el tiempo de ejecución de este algoritmo? Dado que el número de posibles índices a probar es $\binom{n}{2}$ por lo tanto el tiempo de ejecución de este algoritmo es

$(i, j)$

$O(n^2)$

# Problema del subarreglo máximo

- Divide y vencerás



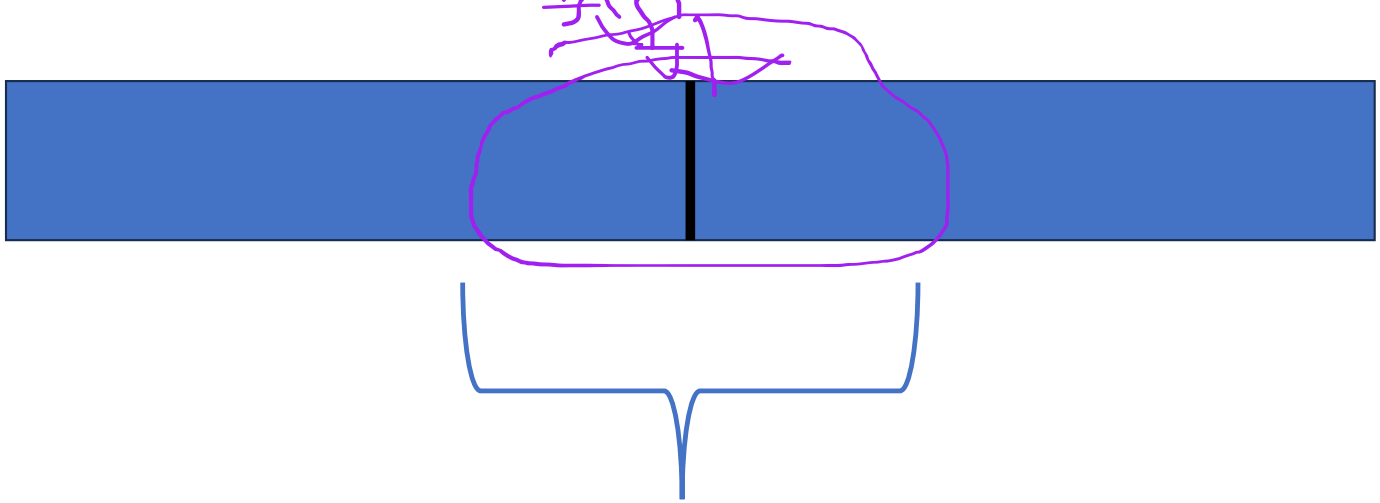

[Cormen, 2022]

[low | mid]

[mid+1 | high]

# Cómo lo implementamos

$$\frac{n}{2} + \frac{n}{2} = n$$



# Problema del subarreglo máximo

MAXIMOSUBARREGLOCRUCE (A,low,mid, high)

```
suma-izquieda=−∞
suma=0
for i=mid downto low
  suma=suma+A[i]
  if suma> suma-izquierda. then
      suma-izquierda=suma
      max-izquierda=i
suma-derecha=−∞
suma=0
for j=mid+1 to high
  suma=suma+A[j]
  if suma> suma-derecha. then
      suma-derecha=suma
      max-derecha=j
return (max-izquierda, max_derecha, suma-izquierda+suma-derecha)
```

# Problema del subarreglo máximo

MAXIMOSUBARREGLO(A,low, high)

1. if high==low then
2. return (low, high, A[low])
3. else mid=$\lfloor \frac{low+high}{2} \rfloor$
4. (low-izquierda, high-izquierda, suma-izquierda) = MAXIMOSUBARREGLO(A,low, mid)
5. (low-derecha, high-derecha, suma-derecha) = MAXIMOSUBARREGLO(A,mid+1, high)
6. (low-cruce, high-cruce, suma-cruce) = MAXIMOSUBARREGLOCRUCE (A,low,mid, high)
7. if suma-izquierda>suma-derecha and suma-izquierda> suma-cruce then
8. return (low-izquierda, high-izquierda, suma-izquierda)
9. if suma-derecha>suma-izquierda and suma-derecha> suma-cruce then
10. return (low-derecha, high-derecha, suma-derecha)
11. else return (low-cruce, high-cruce, suma-cruce)

# Problema del subarreglo máximo

```python
def max_crossing_sum(arr, left, right, mid):
    # Find the maximum sum of the subarray that crosses the midpoint
    left_sum = float('-inf')
    right_sum = float('-inf')

    # Find maximum sum in the left half (starting from mid going left)
    current_sum = 0
    for i in range(mid, left - 1, -1):
        current_sum += arr[i]
        left_sum = max(left_sum, current_sum)

    # Find maximum sum in the right half (starting from mid+1 going right)
    current_sum = 0
    for i in range(mid + 1, right + 1):
        current_sum += arr[i]
        right_sum = max(right_sum, current_sum)

    # Return the sum of both the left and right parts
    return left_sum + right_sum
```

Figura 5a MaximoSubarregloCruce

```python
def max_subarray_sum(arr, left, right):
    if left == right:
        # Base case: only one element
        return arr[left]

    # Find the midpoint of the array
    mid = (left + right) // 2

    # Recursively find the maximum subarray sum for the left, right, and crossing part
    left_sum = max_subarray_sum(arr, left, mid)
    right_sum = max_subarray_sum(arr, mid + 1, right)
    crossing_sum = max_crossing_sum(arr, left, right, mid)

    # Return the maximum of the three sums
    return max(left_sum, right_sum, crossing_sum)
```

Figura 5b. Maximo subarreglo

# Ejercicio

- Implementar el subarreglo máximo utilizando fuerza bruta y divide y vencerás y probarlo con instancias aleatorias de tamaño
  - 100
  - 1000
  - 10000
- Incluir el número de pasos que tarda el algoritmo

- Brute_force
- Sumacruce

# Multiplicación de matrices

MULTIPLICACIONMATRICESCUADRADAS (A,B)

```
n=A.rows
Sea C una nueva matriz de N × N
for i=1 to N
    for j=1 to N
        c_{i,j} = 0
        for k=1 to N
            c_{i,j} = c_{i,j} + a_{i,k} b_{k,j}
return C
```

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \qquad \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix}$$

$c_{11} \leftarrow 0$

$c_{11} = 0 + 3 + 4 + 3$

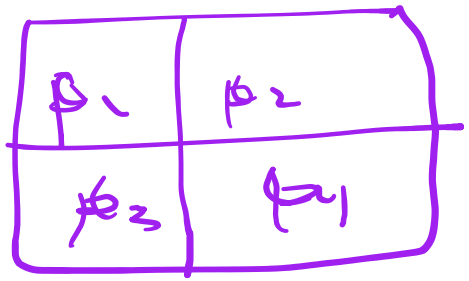
$c_{11} = 10$

$N^2$ (N operaciones) $\in O(N^3)$

$O(n^3)$

# Multiplicación de matrices

Procesados




$O(n^2)$

MULTIPLICACIONMATRICESCUADRADASRECURSIVO (A,B)

1. n=A.rows
2. Sea C una nueva matriz de N × N
3. **If** $n == 1$ **then**
4. $c_{1,1} = a_{1,1}b_{1,1}$
5. **else** particionar A,B, C como se describe en los párrafos anteriores
6. $C_{1,1}$ = MULTIPLICACIONMATRICESCUADRADASRECURSIVO($A_{1,1}, B_{1,1}$)+ MULTIPLICACIONMATRICESCUADRADASRECURSIVO($A_{1,2}, B_{2,1}$)
7. $C_{1,2}$ = MULTIPLICACIONMATRICESCUADRADASRECURSIVO($A_{1,1}, B_{1,2}$) MULTIPLICACIONMATRICESCUADRADASRECURSIVO($A_{1,2}, B_{2,2}$)
8. $C_{2,1}$ = MULTIPLICACIONMATRICESCUADRADASRECURSIVO($A_{2,1}, B_{1,1}$)+ MULTIPLICACIONMATRICESCUADRADASRECURSIVO($A_{2,2}, B_{2,1}$)
9. $C_{2,2}$ = MULTIPLICACIONMATRICESCUADRADASRECURSIVO($A_{2,1}, B_{1,2}$) + MULTIPLICACIONMATRICESCUADRADASRECURSIVO($A_{2,2}, B_{2,2}$)
10. **return** C

$P_1 \rightarrow$ 6.

$P_2 \rightarrow$ 7.

$P_3 \rightarrow$ 8.

$P_4 \rightarrow$ 9.

$A_{11}$ $A_{11}$

$A_{21}$ $A_{11}$

$B_{11}$ $B_{11}$

$B_{21}$ $B_{21}$

# Multiplicación de matrices

- Strassen

La idea del algoritmo de Strassen consiste en hacer la recurrencia un poco más rápido que con el esquema presentado en el algoritmo anterior. La idea medular del algoritmo consiste en que en lugar de realizar 8 multiplicaciones de matrices de $\frac{n}{2} \times \frac{n}{2}$ , realizar solamente 7. Los pasos del algoritmo de Strassen son los siguientes.

# Algoritmo

$N = 2^k$

- 1. Si n=1 cada matriz es un escalar. Por lo pronto es una multiplicación simple. En caso contrario, particiona $A, B$ y la matriz $C$

$$A_{N\times N} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

$$B_{N\times N} = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

# Algoritmo

Cormen, 2022 ← Divide and Conquer

- 2. Calcula las matrices $S_1, S_2, \ldots, S_{10}$. Posteriormente crea 7 matrices de tamaño $\frac{N}{2} \times \frac{N}{2}$: $P_1, P_2, \ldots, P_7$ e inicializalas en cero

$$
\begin{aligned}
S_1 &= B_{1,2} - B_{2,2} \\
S_2 &= A_{1,1} + A_{1,2} \\
S_3 &= A_{2,1} + A_{2,2} \\
S_4 &= B_{2,1} - B_{1,1} \\
S_5 &= A_{1,1} + A_{2,2} \\
S_6 &= B_{1,1} + B_{2,2} \\
S_7 &= A_{1,2} - A_{2,2} \\
S_8 &= B_{2,1} + B_{2,2} \\
S_9 &= A_{1,1} - A_{2,1} \\
S_{10} &= B_{1,1} + B_{2,2}
\end{aligned}
$$

$O(N^2)$

# Algoritmo

- 3. Usar las matrices calculadas en el paso anterior para calcular las submatrices $P_1, P_2, \ldots, P_7$

$O\left(\frac{n}{2}\right)^3$

$n/2$ $n/2$ $n/2$ $n/2$ $n/2$ $n/2$ $n/2$ $n/2$

$$
\begin{aligned}
P_1 &= A_{11} \cdot S_1 \quad (= A_{11} \cdot B_{12} - A_{11} \cdot B_{22}) \,, \\
P_2 &= S_2 \cdot B_{22} \quad (= A_{11} \cdot B_{22} + A_{12} \cdot B_{22}) \,, \\
P_3 &= S_3 \cdot B_{11} \quad (= A_{21} \cdot B_{11} + A_{22} \cdot B_{11}) \,, \\
P_4 &= A_{22} \cdot S_4 \quad (= A_{22} \cdot B_{21} - A_{22} \cdot B_{11}) \,, \\
P_5 &= S_5 \cdot S_6 \quad (= A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22}) \,, \\
P_6 &= S_7 \cdot S_8 \quad (= A_{12} \cdot B_{21} + A_{12} \cdot B_{22} - A_{22} \cdot B_{21} - A_{22} \cdot B_{22}) \,, \\
P_7 &= S_9 \cdot S_{10} \quad (= A_{11} \cdot B_{11} + A_{11} \cdot B_{12} - A_{21} \cdot B_{11} - A_{21} \cdot B_{12}) \,.
\end{aligned}
$$

# Algoritmo

- 4. Calcular $C_{11}, C_{12}, C_{21}, C_{22}$

$$C_{11} = P_4 + P_5 + P_6 - P_2$$
$$C_{12} = P_1 + P_2$$
$$C_{21} = P_3 + P_4$$
$$C_{22} = P_1 + P_5 - P_3 - P_7$$

# Ejercicio

- Utilizar la implementación de código subida (strassen.py) o una propia y realizar lo siguiente
  - Crear una instancia de 64 x 64 y probar
    - Tiempo de ejecución con
      - Método tradicional
      - Método recursivo tradicional
      - Método de Strassen
      - Reportar los resultados

System.time( )